



Data Foundations

AICB-P1 · Ngày 7 · Embedding, Chunking & Vector Store

Tên Giảng Viên

VinUniversity · Phase 1 · Tuần 1 · 2026

HÃY SUY NGHĨ...

“Agent trả lời sai vì model yếu, hay vì nó không có đúng dữ liệu để suy luận?”

Giữ câu hỏi này trong đầu khi học bài hôm nay

1. Data strategy & agent memory
2. Lịch sử: từ TF-IDF đến embedding
3. Embeddings — bản chất
4. Embedding model landscape 2026
5. **Document extraction** (PDF, Excel, HTML...)
6. Chunking & chuẩn bị tài liệu
7. Vector store internals (ANN)
8. FAISS, ChromaDB & landscape
9. Metadata filter & hybrid search
10. Frontier 2025–26
11. Đo lường, chi phí & failure modes
12. Bảo mật & quyền riêng tư
13. Lab 7 + Key takeaways

- Phân biệt được **knowledge data, operational data, contextual data**
- Hiểu **embedding** là lớp biểu diễn nghĩa — cơ chế, cách huấn luyện, và giới hạn
- **Bóc được text ra khỏi file thật** — PDF, Excel, HTML — và biết cái gì bị mất im lặng
- Chọn được **chunking strategy** và giải thích được đánh đổi của nó
- Giải thích được **ANN index** (IVF, PQ, HNSW) đủ để chỉnh tham số, không chỉ gọi API
- Nhận diện được các **failure mode im lặng** — lỗi không ném exception nhưng phá recall
- Build được một **mini retrieval integration** nối agent với dữ liệu riêng

Artifact pack cần nộp

Data inventory + chunking / embedding script + vector store index + semantic search demo + retrieval-enabled answer function

- 1 bộ dữ liệu mẫu đã được chunk và index
- 1 script truy vấn semantic search có trả kết quả liên quan
- 1 hàm trả lời sử dụng context retrieve được thay vì hỏi LLM “chay”
- 1 bảng đo **recall@5** trên tối thiểu 10 câu hỏi tự sinh

Data Strategy Cho Sản Phẩm AI

Khi ai cũng gọi được model mạnh qua API, câu hỏi đã đổi từ “dùng model nào? sang “agent được phép biết gì, và có đúng dữ liệu để suy luận không?

Dữ liệu bẩn / thiếu

- PDF scan lỗi OCR
- Policy cũ, chưa cập nhật
- Chunk cắt giữa câu
- Không có metadata

Kết quả: agent hallucinate, trả lời sai, user mất niềm tin.

Dữ liệu sạch / đầy đủ

- Text đã chuẩn hóa, metadata đầy đủ
- Nguồn rõ ràng, có version
- Chunk theo section hợp lý
- Filter được theo category + freshness

Kết quả: retrieve đúng, answer grounded, có trích nguồn.

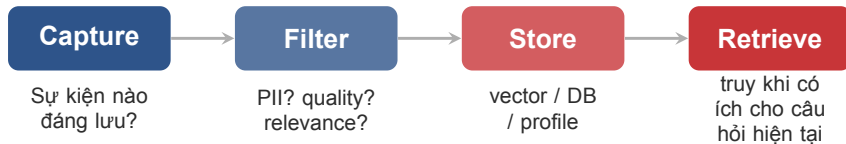
Loại data	Đặc điểm	Ví dụ	Retrieval fit
Knowledge	Ít thay đổi, dạng text dài, cần chunk + embed	FAQ, SOP, chính sách, hợp đồng, tài liệu kỹ thuật	Rất cao — lý tưởng cho vector store
Operational	Thay đổi liên tục, dạng structured (SQL / JSON / logs)	Trạng thái đơn hàng, CRM, ticket, tồn kho	Thấp — dùng function calling / SQL, không embed
Contextual	Gắn với session / user hiện tại, ngắn gọn	User profile, lịch sử hội thoại gần nhất, giỏ hàng	Trung bình — inject trực tiếp, ít khi cần semantic search

Knowledge data phù hợp retrieval; operational data cần query có kiểm soát; contextual data nên inject ngắn và đúng lúc

Governance trước khi index: ai sở hữu & cập nhật dữ liệu · ai được truy cập (ACL vs public nội bộ) · bao lâu re-index · PII có cần mask không — ☐ không “cứ nạp hết vào vector DB đã”.

Loại PII	Ví dụ	Kỹ thuật mask	Rủi ro nếu bỏ qua
Tên cá nhân	“Nguyễn Văn A”	Thay bằng [PERSON]	Trung bình
Số điện thoại	“0912-xxx-xxx”	Regex replace	Cao
Email	“user@email.com”	Hash hoặc remove	Cao
CMND / CCCD	“012345678901”	Xóa hoàn toàn	Rất cao
Địa chỉ	“123 Lê Lợi, Q.1”	Generalize thành “Q.1, HCM”	Trung bình

Mask trước khi embed — không bao giờ lưu raw PII trong vector store. *Vector không phải dữ liệu đã ẩn danh — embedding có thể bị đảo ngược gần đúng nguyên văn (Morris et al., EMNLP 2023; ALGEN 2025). Đầy đủ ở §11 — Bảo mật & Compliance.*

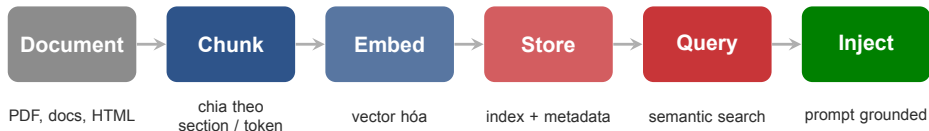


KHÔNG tự động là

memory: prompt dài hơn · file PDF upload một lần không truy lại có chủ đích · toàn bộ chat history · “lưu cho chắc” — những thứ này thường tạo nhiều hơn là hữu ích.

Khung nghĩ đúng — và đừng nhầm với retrieval

Memory là **data + policy + retrieval**; thiếu một trong ba thì hệ thống khó ổn định. **Retrieval** tìm context cho câu hỏi hiện tại (relevance, grounding); **memory** giữ trạng thái người dùng qua thời gian (continuity). Nhầm hai khái niệm là lý do agent “quên” context vừa retrieve ở lượt sau. Vocab chuẩn: **working / episodic / semantic / procedural**.



Đây là trục xương sống của cả Ngày 7

Mọi phần tiếp theo hôm nay chỉ đào sâu **một mắt xích** trong pipeline này: **Chunk** → phần Chunking, **Embed** → phần Embeddings, **Store** → phần Vector Store (ChromaDB/FAISS) và ANN internals, **Query** → phần Retrieval & Hybrid Search, **Inject** → phần Kết nối Agent với Data và Eval.

Lịch Sử: Từ TF-IDF Đến Embedding

Embedding + cosine similarity là ý tưởng từ 1975 — cái thay đổi là vector đến từ đâu, không phải hình học

- Lexical search (TF-IDF, BM25) chỉ khớp khi **đúng từ** xuất hiện ở cả query lẫn document.
- **IDF** (Spärck Jones, 1972): từ hiếm được tính trọng số cao hơn từ phổ biến — nền tảng của TF-IDF.
- **BM25** (Robertson & Spärck Jones, giới thiệu tại TREC-3, **1994**) — vẫn là baseline lexical chuẩn mực đến 2026.

Ví dụ thất bại

Query: “*chính sách hoàn tiền*”. Document chỉ viết: “*quy định đổi trả sản phẩm*”. Không từ nào trùng khớp $\Rightarrow$ BM25/TF-IDF không tìm ra, dù nghĩa gần như giống hệt.

Lưu ý: BM25 không “lỗi thời”: BEIR (2021, 18 dataset) cho thấy đây vẫn là baseline mạnh — một dense model fine-tune trên MS MARCO có thể **thua** BM25 khi ra ngoài domain huấn luyện.

Năm	Cột mốc	Ý nghĩa
1972	Spärck Jones — IDF	Từ hiếm đáng giá hơn từ phổ biến
1975	Salton — Vector Space Model	Văn bản/query = vector, so bằng hình học
1990	Deerwester — LSA/LSI	SVD nén còn ~100 chiều “khái niệm” — tổ tiên của dense embedding
1994	Robertson — BM25 (TREC-3)	Baseline lexical chuẩn mực đến hôm nay
2013	Mikolov — word2vec	Dense word vector đầu tiên ở quy mô web
2016	Malkov & Yashunin — HNSW	Graph ANN — default index của hầu hết vector store hôm nay
2018/19	Devlin — BERT	Contextual encoder, giới hạn 512 token
2019	Reimers & Gurevych — SBERT	Sửa hình học similarity mà BERT thô không làm được
2020	Karpukhin — DPR	Dense retrieval vượt BM25 (+9 đến +19% top-20 accuracy)
2025–26	Decoder-LLM embedder + MRL + quantization	“Table stakes”: Qwen3-Embedding, Gemini Embedding 2, Voyage 4

Bỏ bớt các mốc phụ để giữ một trang; chi tiết từng mốc nằm ở các frame sau và trong RESEARCH companion

Cross-encoder: BERT gốc

So hai câu $\Rightarrow$ BERT (2018/19) — 512-token cap — cần joint attention.

- Muốn so hai câu $\Rightarrow$ phải đưa **cả cặp** qua BERT cùng lúc.
- So khớp giữa 10.000 câu $\Rightarrow \sim 50$ triệu phép suy luận.
- **~ 65 giờ** trên GPU để tìm cặp giống nhau nhất.

Train cho masked-LM, không cho pooled similarity — không báo lỗi, chỉ cho vector không so sánh được.

Bi-encoder: SBERT (2019)

Reimers & Gurevych (EMNLP 2019): siamese network, contrastive fine-tune trên NLI.

- Encode mỗi câu **một lần**, độc lập $\Rightarrow$ vector cố định, precompute trước.
- So sánh bằng cosine similarity, không cần chạy lại BERT.
- Cùng bài toán: **~ 5 giây** — độ chính xác tương đương trên STS.

Đây là lý do vector store precompute embedding tài liệu một lần rồi query nhanh.

Embeddings — Bản Chất

Embedding không phải phép màu; nó là một hàm học được, và hình học của nó là sản phẩm phụ của mục tiêu huấn luyện

03

Embedding — Hàm *học được* biến dữ liệu thô (text, ảnh, audio) thành **vector số cùng chiều**, sao cho “gần nghĩa” → “gần hình học”.

Một pipeline cụ thể, chạy trên GPU/CPU của ai đó:

1. **Tokenize**: cắt câu thành subword token
2. **Encoder**: token qua nhiều lớp Transformer self-attention → vector *theo ngữ cảnh*
3. **Pooling**: gộp vector token thành **một** vector câu — mean, last-token, hoặc [CLS]

Pooling không trung lập

jina-embeddings-v5: mean pooling (v4) → last-token — mất Late Chunking, vốn cần vector theo token. Đổi pooling là đổi cả model.

Cosine Similarity

$$\cos(\vec{A}, \vec{B}) = \frac{\vec{A} \cdot \vec{B}}{\|\vec{A}\| \|\vec{B}\|}$$

- Tử: tích vô hướng (dot product)
- Mẫu: tích hai độ dài đã chuẩn hoá
- 1 = cùng hướng, 0 = vuông góc, -1 = ngược hướng

Không cần tự code

Hầu hết vector store mặc định dùng cosine — hiểu score 0.87 so với 0.31 nghĩa là gì (frame sau).

Euclidean Distance

$$d(\vec{A}, \vec{B}) = \sqrt{\sum_{i=1}^n (A_i - B_i)^2}$$

- Khoảng cách “đường chim bay” n chiều
- 0 = trùng nhau, càng lớn = càng xa

Cặp 1

$$\vec{A} = [1, 2, 3]$$

$$\vec{B} = [2, 4, 6]$$

$$\cos(\vec{A}, \vec{B}) = ?$$

Gợi ý: $\vec{A} \cdot \vec{B} = 1 \times 2 + 2 \times 4 + 3 \times 6$

Cặp 2

$$\vec{C} = [1, 0, 0]$$

$$\vec{D} = [0, 1, 0]$$

$$\cos(\vec{C}, \vec{D}) = ?$$

Gợi ý: hai vector này có điểm chung nào không?

Tính trên giấy hoặc máy tính (3 phút), so đáp án với người bên cạnh.

Lưu ý: Cặp 1 có cosine = 1.0 dù $\vec{B} = 2\vec{A}$. Vì sao? Điều này nói gì về cosine similarity so với Euclidean distance?

Lưu ý: Steck, Ekanadham & Kallus (Netflix+Cornell), *Is Cosine-Similarity of Embeddings Really About Similarity?*, WWW 2024: cosine similarity của embedding đã học “**can yield arbitrary and meaningless similarities**” — với linear model regularized, cosine **không xác định duy nhất**.

Nguồn: *arXiv:2403.05440, WWW'24*.

- Regularization deep learning tác động “implicit và unintended” lên cosine.
- Một số trường hợp, cosine tệ hơn dot product chưa chuẩn hoá.

Cách dạy đúng

Cosine là **convention** hiệu quả, không phải **sự thật** về ý nghĩa. “Metric mặc định” là lựa chọn kỹ thuật, không phải luật tự nhiên.

Symmetric

- Query và document **cùng loại** (câu ↔ câu)
- Ví dụ: tìm câu trùng lặp, STS

Asymmetric

- Câu hỏi **ngắn** tìm đoạn văn **dài**
- Đây chính là RAG

Model được huấn luyện khác nhau cho hai phía — nên expose **prefix** hoặc **instruction** riêng: E5 dùng query: / passage:; Nomic v2 dùng search_query: / search_document:.

Lưu ý: Bỏ prefix **không báo lỗi** — nó âm thầm tạo ra embedding lệch calibration, xếp hạng sai. Model card Qwen3-Embedding-8B: dùng instruction cải thiện **1% đến 5%** so với không dùng.

```
# pin the version -- 5.6.1 shipped one week before this lecture
from sentence_transformers import SentenceTransformer
from sentence_transformers.util import cos_sim

model = SentenceTransformer("BAAI/bge-m3")

texts = ["Chinh sach hoan tien", "Quy dinh doi tra"]
embeddings = model.encode(texts, normalize_embeddings=True)

score = cos_sim(embeddings[0], embeddings[1])
print(score.item())
```

`normalize_embeddings=True` đã chuẩn hoá L2 ngay trong `.encode()` — nên `cos_sim` ở đây tương đương cosine, không lệch bởi magnitude.

Bức Tranh Embedding Model 2026

Không có model “tốt nhất”; chỉ có model đúng cho trục quality/speed/size/cost mà bạn cần

04

Model	Params	Output dims	Max input	License
Qwen3-Embedding (0.6B/4B/8B)	0.6–8B	tới 4096, MRL → 32	32K (cả 3 size)	Apache-2.0
EmbeddingGemma	308M	768, MRL → 128	2K	Gemma terms
BGE-M3	~568M	dense + sparse + multi-vec	8192	MIT
Nomic Embed Text v2 (MoE)	475M/305M active	768, MRL → 256	512	Apache-2.0
Jina Embeddings v4	3.8B	2048 (hoặc multi-vector)	long-context	—

Số liệu verbatim từ HF model card / arXiv của từng model, chốt 2026-07-30. BGE-M3 tạo cả ba biểu diễn dense+sparse+multi-vector cùng lúc — hybrid retrieval SOTA là một model, không phải ba hệ thống ghép lại. Nomic v2 max input chỉ 512 token, ngắn hơn nhiều embedder cũ dù là model 2025.

Model	Dims	Max input	Giá /1M token input
OpenAI text-embedding-3-large	tới 3072	8191	\$0.13
OpenAI text-embedding-3-small	tới 1536	8191	\$0.02
Google gemini-embedding-2	MRL native	8192	\$0.20 (\$0.10 batch)
Voyage voyage-3.5	2048/1024/512/256	512	\$0.06
Cohere embed-v4	256/512/1024/1536	1024	giá chưa xác minh được

Giá xác minh trên trang chính thức từng vendor, 2026-07-30. Không có tier giá batch chính thức — chỉ “khoảng nửa giá” qua Batch API, không có số cụ thể.

Lưu ý: Lầm tưởng: “OpenAI embeddings là mặc định tốt nhất.” -3-large/-small phát hành 25/1/2024, chưa cập nhật ~2.5 năm trong khi Google/Voyage/Jina ra nhiều thế hệ mới. -3-large: \$0.13/M so với voyage-3.5: \$0.06/M — không có bằng chứng vượt trội. gemini-embedding-001 (giới hạn 2K token) đã bị thay bởi -2.

MTEB đã tách thành nhiều board **không so sánh được với nhau**: MTEB(Eng, v2), MTEB(Multilingual)/MMTEB, MTEB(Code)... Điểm v2 không so được với v1.

Ví dụ thật, **cùng một model** (Gemini Embedding), ba con số:

- MTEB(Multilingual) Mean(Task): **68.32** — con số được quảng bá làm headline
- MTEB(Eng, v2) Mean(Task): **73.28**
- Task-Type Mean: 59.64

Lưu ý: Lầm tưởng: “68.32 là điểm MTEB tiếng Anh.” Sai — đó là điểm **MULTILINGUAL**. Điểm English v2 thật là 73.28. Lỗi này lan qua nhiều trang tổng hợp, tạo ra so sánh tự mâu thuẫn (vd. đặt jina-v5-small 71.7 “vượt” Gemini 68.32, trong khi English thật của Gemini là 73.28).

Quy tắc cho lớp

Một con số MTEB **vô nghĩa** nếu thiếu board + version + aggregation + ngày. (Cập nhật: từ 2025–26 MTEB đã chuyển sang kết quả **verified**, không còn thuần self-reported.)

- **VN-MTEB (EACL 2026 Findings)**: benchmark embedding tiếng Việt chuẩn hóa đầu tiên — 41 dataset, 6 loại task (retrieval, reranking, classification, clustering, pair classification, STS).
- Phát hiện đáng chú ý: model dùng **RoPE** vượt trội hơn model dùng absolute positional embedding trên task tiếng Việt, ở nhóm model cùng quy mô.
- Trước VN-MTEB, nhóm phát triển thường chọn model tiếng Việt theo điểm MTEB **tiếng Anh** và hy vọng transfer tốt — không đảm bảo.

Model chuyên biệt tiếng Việt

AITeamVN/Vietnamese_Embedding **v2**: fine-tune từ BGE-M3 trên ~ 1.1 triệu triplet (query, positive, negative) tiếng Việt; 2048 max sequence, 1024 dims, Apache-2.0. Đường đi thực dụng: không dùng thẳng model đa ngôn ngữ, cũng không train từ đầu — **fine-tune model đa ngôn ngữ mạnh trên domain triplet**.

5 trục quyết định, không phải 1 thứ hạng leaderboard: deployment, max input, dimension/precision, ngôn ngữ, query shape, license.

Quy trình 20 phút:

1. Viết **độ dài chunk tối đa** và **dạng query** (có exact code/SKU/ID không?) — loại bớt ứng viên trước khi benchmark.
2. Lập shortlist 2–3 model theo **license + deployment** (on-device/air-gapped hay API được phép?).
3. Xây bộ eval 50–100 query từ chính corpus của bạn — **không** chỉ dựa MTEB.
4. Đo recall@k trên bộ eval, dùng đúng prefix/instruction cho từng model.
5. Chỉ sau đó mới tinh chỉnh dimension và quantization (MRL, int8/binary).

2 lưu ý nhanh sau khi chọn

(1) **SKU/code trong query:** dense embedding thuần blur token chính xác — cần sparse (BGE-M3 có sẵn) hoặc hybrid BM25 (§9). (2) **Đa phương thức:** Cohere embed-v4 / Google gemini-embedding-2 nhúng text+image(+audio/video) vào cùng một vector space — vẫn áp dụng đủ 4 trục + license; Lab 7 vẫn dùng embedding text thuần.

Document Extraction: Từ File Thật Đến Text

Trước khi có chunk, có embedding, có vector store — bạn phải lấy được text ra khỏi file. Đây là khâu quyết định trần chất lượng của cả pipeline

Nhóm	Ví dụ	Cách xử lý đúng
Unstructured	PDF scan, ảnh, chữ viết tay, audio transcript	OCR / VLM parsing → text + layout, rồi chunk theo cấu trúc
Semi-structured	HTML, DOCX, PPTX, Markdown, email	Bóc boilerplate, giữ cây heading → chunk theo heading
Structured	Excel, CSV, SQL table, JSON, log	Thường KHÔNG nên embed thô — text-to-SQL hoặc serialize theo hàng (§5, cuối section)

Ba nhóm cần ba đường xử lý khác nhau — đừng ép tất cả qua cùng một parser

PDF là định dạng mô tả CÁCH VẼ trang, không mô tả NỘI DUNG. Nó lưu “đặt glyph này tại toạ độ (x,y)” — không lưu “đây là ô thứ 3 của hàng thứ 2 trong bảng”.

- **Born-digital vs scanned:** file sinh từ Word có sẵn text layer; file scan chỉ là ảnh $\Rightarrow$ bắt buộc OCR.
- **Reading order:** 2 cột, sidebar, chú thích — pdftotext đọc theo thứ tự vẽ, có thể trộn cột trái với cột phải thành câu vô nghĩa.
- **Header/footer lặp:** tên công ty + số trang chèn vào giữa mọi chunk, làm nhiễu embedding.
- **Bảng:** mất quan hệ hàng–cột là lỗi tồn kém nhất (frame riêng ở sau).
- **Công thức, biểu đồ, hình:** thông tin nằm trong pixel, không có trong text layer.

Lưu ý: “PDF là text, chỉ cần pdftotext” — đúng với đúng một loại tài liệu: born-digital, một cột, không bảng. Với corpus thật, đây là giả định sai đỗi nhất trong cả pipeline.

Công cụ	Loại	Ghi chú thực dụng
Docling (IBM)	Pipeline, MIT license	DocLayNet layout + TableFormer; mạnh về bảng phức tạp; ra Markdown/JSON
MinerU	Pipeline hoặc VLM	Bản 2.5-Pro đứng đầu OmniDocBench v1.6 theo báo cáo của chính nhóm tác giả
Marker (Datalab)	Pipeline	Nhanh; benchmark v2 do chính Datalab chạy
Unstructured	Pipeline, hosted	30+ định dạng (kể cả email, HTML); có sẵn chunking
LlamaParse	Hosted	Trả phí theo trang; tiện khi không muốn tự vận hành
olmOCR (AI2)	VLM 7B	Chuyên linearize PDF cho data pipeline; 82.4 trên olmOCR-Bench
MarkItDown (MS)	Chuyển đổi nhẹ	Office-heavy, không GPU; hợp prototype, yếu với PDF scan

Nguồn: *Docling* [arXiv:2501.17887](#) · *olmOCR* [github.com/allenai/olmocr](#) · *dots.mocr* [arXiv:2512.02498](#) · *DeepSeek-OCR* [arXiv:2510.18234](#)

OmniDocBench (CVPR 2025, 1.355 trang, 9 loại tài liệu) chấm 4 trục: **text** (edit distance), **công thức** (CDM), **bảng** (TEDS), **reading order**.

- Trên v1.5: **GLM-OCR 94,6%** (SOTA), **PaddleOCR-VL-1.5 >94%**, **Gemini 3 Pro 90,3%**.
- MinerU2.5-Pro báo cáo **95,69** trên v1.6, Table TEDS **93,42** — con số từ chính paper của nhóm tác giả.

Lưu ý: Khi nhiều hệ vượt 94%, phần tăng thêm chủ yếu là “vá edge case”, không còn phản ánh chất lượng thực tế trên corpus của *bạn*. Tệ hơn: các bảng xếp hạng **mâu thuẫn nhau** — cùng bộ công cụ, đổi bộ tài liệu là đổi thứ hạng. Và phần lớn benchmark được chạy bởi chính nhà cung cấp công cụ.

Việc cần làm thay vì tin bảng xếp hạng

Lấy **20 trang khó nhất** trong corpus của bạn (scan mờ, bảng lồng, 2 cột), chạy qua 2–3 công cụ, và **đọc bằng mắt**. Đó là benchmark duy nhất có giá trị quyết định.

Menu, banner, ad, footer, “bài liên quan” — nếu embed thẳng HTML thô, phần lớn vector mô tả **giao diện**, không phải nội dung.

- **Trafilatura** — pipeline heuristic nhiều tầng, **không ML, không GPU**, khoảng **14–22 ms/trang**. Mặc định hợp lý cho quy mô lớn.
- **ReaderLM-v2** (Jina) — transformer **1,54B** huấn luyện riêng cho HTML→Markdown: cấu trúc trung thực hơn, nhưng cần GPU và chậm hơn nhiều bậc.
- **justext** — bóc boilerplate theo mật độ stopword ở mức đoạn văn.
- Trang đã convert đúng thường dùng **ít hơn khoảng 65% token** so với HTML thô ⇒ giảm thẳng chi phí embed.

Chiến lược hai tầng thực dụng

Chạy trafilatura trước cho toàn bộ corpus; chỉ chuyển sang parser nặng (ReaderLM / html-to-markdown) cho những trang mà cấu trúc thực sự quan trọng. Đừng trả giá GPU cho 100% corpus để cứu 5% trang.

- **DOCX** — giữ được cây heading (rất quý cho chunking); **mất** comment, tracked changes, footnote nếu parser không xử lý riêng. Một hợp đồng mà phần thương lượng nằm ở comment thì bản parse là bản *sai*.
- **PPTX** — text trong shape thường rời rạc, thứ tự đọc theo thứ tự tạo shape chứ không theo thị giác; **speaker notes** thường là phần có giá trị nhất và thường bị bỏ quên.
- **Email** — chữ ký, disclaimer pháp lý và thread reply lồng nhau khiến cùng một đoạn văn bị index **hàng chục lần** $\Rightarrow$ near-duplicate làm hỏng top- k .

Quy tắc

Với mỗi định dạng, hỏi hai câu: **(1)** cấu trúc nào đáng giữ để chunk theo? **(2)** nội dung nào bị mất im lặng khi convert? Câu hai quan trọng hơn — vì không có exception nào được ném ra.

Sai lầm phổ biến: coi mỗi sheet là một bảng sạch và đẩy thẳng vào `pandas.read_excel`.

- **Ô merge** $\Rightarrow$ NaN rải rác; phải **fill-down** để khôi phục quan hệ hàng.
- **Header nhiều tầng** (2–3 dòng) $\Rightarrow$ tên cột thật là *ghép* của các tầng: “Q2 2026 · Doanh thu · VND”.
- Một sheet có thể chứa **nhiều bảng rời** + ô ghi chú tự do; ranh giới bảng phải tự dò.
- **Formula vs value**: lưu công thức hay kết quả? Với retrieval, gần như luôn là **kết quả**.
- Số, ngày tháng, đơn vị: định dạng hiển thị khác giá trị thật (1.234,56 vs 1234.56).

Lưu ý: Định dạng serialize quyết định recall. Một hàng nên trở thành một đơn vị **tự đủ nghĩa**:
“Q2 2026 | Doanh thu | 4,2 trd VND” — không phải một ô “4.2” trôi nổi không có header.

Khi chunker cắt một bảng theo ký tự, quan hệ hàng–cột biến mất: header “*Doanh thu Q2 2026*” rơi vào chunk này, giá trị “*4,2 tỷ*” rơi vào chunk khác. Không kỹ thuật retrieval nào ghép lại được.

Bảng chứng định lượng — Structure-aware Tabular Chunking (STC) so với

RecursiveCharacterTextSplitter, trên MAUD (39.231 bản ghi hợp đồng M&A từ SEC EDGAR), ngân sách 512 token:

Chỉ số	Recursive	STC
MRR (hybrid)	0,358	0,595
Recall@1 (hybrid)	0,347	0,539
Recall@1 (BM25)	0,366	0,754
Số chunk sinh ra	—	ít hơn ~40%

Nguồn: Guttal et al., “Structure-Aware Chunking for Tabular Data in RAG”, arXiv:2605.00318 (5/2026).

Với dữ liệu đã nằm trong bảng SQL, vector search thường là công cụ **sai**:

- “Tổng doanh thu quý 2 theo vùng” — cần **aggregation**, không phải similarity. Không embedding nào cộng được số.
- “Đơn hàng mới nhất của khách X” — cần **sort + filter chính xác**, đúng thế mạnh của SQL.
- “Chính sách hoàn tiền nói gì?” — **đây** mới là việc của vector search.

Kiến trúc thực dụng: định tuyến, không chọn một

Một router quyết định: câu hỏi số liệu → **text-to-SQL**; câu hỏi khái niệm → **vector search**; câu hỏi quan hệ → graph. Nhiều hệ production 2026 chạy cả ba song song rồi hợp nhất kết quả.

Lưu ý: Embed toàn bộ bảng giao dịch thành vector là anti-pattern tốn kém và kém chính xác. Trước khi embed bất cứ thứ gì, hỏi: **câu hỏi này có phải câu hỏi ngữ nghĩa không?**

- **Dấu thanh và dấu phụ** mang nghĩa: OCR nhầm một dấu là đổi hẳn từ (*ma / mà / má / mã / mạ*). Tesseract mặc định yếu ở đúng điểm này.
- **Độ phân giải scan tối thiểu 300 DPI** — dưới ngưỡng đó, o/ô/ơ và a/ă/â bắt đầu lẫn.
- **Chuẩn hoá Unicode bắt buộc**: cùng một chữ “ế” có thể mã hoá dạng sẵn (NFC) hoặc tổ hợp (NFD). Hai dạng **không khớp nhau** khi so chuỗi và tạo ra chunk trùng lặp mà mắt thường không phân biệt được. Chuẩn hoá NFC toàn corpus ngay sau khi parse.
- Công cụ chuyên biệt tồn tại (VietOCR, PaddleOCR fine-tune cho tiếng Việt); các VLM đa ngôn ngữ mới cũng đã khá hơn đáng kể.

Nguồn: “A Survey on Vietnamese Document Analysis and Recognition”, arXiv:2506.05061 · Sino-Vietnamese PaddleOCRv5, arXiv:2510.04003.

Parse xong **chưa** phải là xong. Trước khi chunk:

- **Unicode NFC** cho toàn bộ text (đặc biệt quan trọng với tiếng Việt).
- **Bỏ header/footer lặp** — dò chuỗi xuất hiện ở cùng vị trí trên hầu hết trang.
- **Nối từ bị gạch nối cuối dòng** (*de-hyphenation*) và gộp dòng thành đoạn.
- **Xoá trang trắng, mục lục, trang bìa** nếu không mang thông tin truy vấn được.
- **Khử trùng lặp** — cùng một tài liệu thường tồn tại nhiều bản (v1, v2, final, final-2).

Provenance: giữ từ đây, không thể thêm sau

Mỗi đoạn text nên mang theo **tên file, số trang, đường dẫn heading** ngay từ lúc parse. Đây là thứ cho phép câu trả lời trích nguồn “theo trang 14 của hợp đồng A”. Nếu không giữ ở khâu này, không khâu nào sau đó tạo lại được.

Chunking & Chuẩn Bị Tài Liệu

Chunk sai thì mọi retrieval xây trên top- k đều sai theo — không mô hình embedding nào cứu được một chunk tồi

Chunking — Chia tài liệu dài thành đoạn (chunk) nhỏ hơn, embed/index *riêng lẻ* — tránh vượt giới hạn token, giúp retrieval trúng đúng *đoạn* thay vì cả file.

	Chunk quá to	Chunk hợp lý	Chunk quá nhỏ
Kích thước	>1000 tokens	200–500 tokens	<50 tokens
Vấn đề	Dính nhiều chủ đề vào cùng một vector	Một ý / một section trọn vẹn, overlap với chunk liền kề	Mất ngữ cảnh, retrieve nhiều mảnh rời rạc
Hệ quả khi retrieve	Retrieve trúng nhưng inject rất nhiều	Cân bằng precision/completeness	Khó tổng hợp thành câu trả lời đầy đủ

Rule of thumb: bắt đầu đơn giản với chunk theo section/heading, tối ưu sau bằng eval — không đoán cảm tính

BERT (2018) có bảng positional embedding giới hạn cứng ở **512 token** — đây là giới hạn *kiến trúc* của một model cụ thể năm 2018, không phải một quy luật retrieval.

- Con số này sống sót qua vô số tutorial RAG như một “default” bất di bất dịch — lâu hơn hẳn lý do kỹ thuật ban đầu.
- Embedder 2026 đã bỏ xa nó: BGE-M3 / Jina v2–v3 tới 8K token; Qwen3-Embedding tới 32K; Cohere Embed v4 tới 128K.

Lưu ý: Không có ngưỡng “512 token” phổ quát. Bhat, Rudat, Spiekermann & Flores-Herr (arXiv:2505.21700, 2025): chunk **64–128 token** tối ưu cho câu hỏi factoid ngắn; **512–1024 token** tốt hơn khi cần hiểu ngữ cảnh rộng — và tối ưu còn phụ thuộc *embedding model* (Stella lợi với chunk lớn, Snowflake lợi với chunk nhỏ, tập trung entity).

Hệ quả

Đổi embedding model $\Rightarrow$ phải đo lại chunk size. Đừng copy con số của deck khác sang model khác.

Chiến lược	Cách hoạt động	Chi phí	Khi nào dùng
Fixed-size split	Cắt theo số ký tự/token cố định, không quan tâm ranh giới	~Free	Baseline khởi điểm
+ Overlap	Chồng lấn N câu/token giữa các chunk liền kề	~Free	Giảm mất ngữ cảnh tại điểm cắt
Recursive character splitting	Thử tách theo $\backslash n \backslash n \rightarrow \backslash n \rightarrow \text{space} \rightarrow$ ký tự, đệ quy khi vẫn quá dài	~Free	Gần như luôn thắng fixed-size, chuẩn mặc định
Structure-aware	Cắt theo heading, section, bảng, code block	~Free—cheap	Tài liệu có cấu trúc rõ (docs, FAQ, policy)
Semantic (break-point)	Embed từng câu, cắt tại điểm cosine similarity giảm mạnh	1 lượt embed/câu	Chỉ khi đã đo thấy gap thật (xem myth kế tiếp)

Càng lên cao chi phí càng tăng — chỉ leo khi đã đo được một gap retrieval thật

Nhiều tutorial RAG coi semantic (embedding-breakpoint) chunking là upgrade *tự động* so với fixed-size.

Lưu ý: Qu, Tu & Bao (Vectara / UW-Madison / Penn), *Is Semantic Chunking Worth the Computational Cost?*, arXiv:2410.13070, **NAACL 2025 Findings**: chi phí tính toán “**not justified by consistent performance gains**” — trên document retrieval, evidence retrieval, retrieval-based QA.

- Con số “semantic chunking 87% vs fixed-token 50%” (một “clinical study”) **không tồn tại trong nguồn nào** — đừng dùng.
- Con số “chậm hơn $\sim 14\times$ ” là benchmark throughput của **Chonkie**, không phải từ paper — ghi đúng nguồn.

Nguồn

Qu et al., NAACL 2025 Findings (2025.findings-naacl.114) — nhãn “Vectara 2024” và “Qu 2025” là *cùng một paper* bị đếm hai lần.

Late Chunking (Jina, 2024)

Đảo ngược thứ tự: embed **toàn văn bản** bằng long-context model trước, chunk **ngay trước** mean pooling.

- Chunk vector vẫn giữ ngữ cảnh toàn tài liệu (vd. resolve pronoun xuyên ranh giới chunk).
- Không cần fine-tune riêng, chạy với bất kỳ long-context embedder nào.

arXiv:2409.04701 (Günther et al.)

*Phụ thuộc mean pooling — Jina v5 đổi sang last-token pooling nên **mất** khả năng này.*

Contextual Retrieval (Anthropic, 2024)

Prepend 50–100 token ngữ cảnh do LLM sinh vào **mỗi** chunk, trước khi embed và index BM25.

- Top-20 failure rate: 5.7% (baseline) → 3.7% (−35%, +contextual embed) → 2.9% (−49%, +BM25) → 1.9% (−67%, +rerank).
- Chi phí: \$1.02/triệu token tài liệu (prompt caching).

Lưu ý: eval riêng của Anthropic (vendor). Reproduction độc lập (Merola & Singh, ECIR 2025): NDCG@5 0.317 vs 0.312 — thật nhưng nhỏ hơn nhiều so với 49%.

Model	Max input
Nomic Embed Text v2 MoE	512
mxbai-embed-large	~512
EmbeddingGemma	2,048
gemini-embedding-001	2,048
BGE-M3 / Arctic-Embed 2.0 / nomic-embed-text-v1.5 / Jina v2-v3	8,192
Qwen3-Embedding (0.6B / 4B / 8B)	32,768 (cả 3 size)
jina-embeddings-v5-text	32K
Cohere Embed v4	128K

Lưu ý: Text vượt `max_seq_len` bị cắt **âm thầm** bởi hầu hết client library — không raise lỗi. Không có bản Qwen3-Embedding 40K; model card ghi rõ 32K cho cả ba size.

Chunk xấu (raw, không section)

Query: “Chính sách đổi trả áp dụng trong bao lâu?”

Retrieved (cosine 0.61): “...giao hàng miễn phí đơn trên 500k. Đổi trả trong 30 ngày. Liên hệ hotline 1900...”

LLM answer: “Bạn có thể đổi trả và liên hệ hotline 1900...” — *nhiều, thiếu chi tiết*

Chunk tốt (theo section + metadata)

Query: “Chính sách đổi trả áp dụng trong bao lâu?”

Retrieved (cosine 0.89): “Chính sách đổi trả: khách hàng có 30 ngày kể từ ngày nhận hàng để yêu cầu đổi trả. Sản phẩm phải còn nguyên tem.”

LLM answer: “30 ngày kể từ ngày nhận, sản phẩm còn nguyên tem.” — *chính xác, có nguồn*

Bên Trong Vector Store: Thuật Toán ANN

Vector store không “tìm kiếm ma thuật” — nó đánh đổi recall, latency và memory theo những cách rất cụ thể

Mỗi record lưu id + vector + document + metadata — phần còn lại của section này chỉ thay đổi CỘT vector.

Exact k-NN: $O(N \cdot d)$ mỗi query — với $N=10$ triệu, $d=1536$: **~15 tỷ** phép nhân–cộng cho MỘT query.

Recall

Tìm đúng láng giềng
thật hay không

Latency

Trả lời trong bao lâu

Memory

Index chiếm bao
nhiêu RAM/disk

Nguyên lý xuyên suốt section

Mọi kỹ thuật ANN chỉ là một cách **không nhìn hết corpus**. Mỗi index tiêu một trong ba đồng tiền trên để mua đồng tiền còn lại — không index nào thắng cả ba.

- **Cơ chế:** lưu mọi vector nguyên bản (uncompressed); tính khoảng cách tới TẤT CẢ; sắp xếp. FAISS: IndexFlatL2 / IndexFlatIP.
- **Recall: 100% theo định nghĩa** — đây là **ground truth** để đo recall của mọi index khác.
- **Memory:** $N \times d \times 4$ bytes (float32). $N=10\text{M}$, $d=1536 \Rightarrow \sim 61.4 \text{ GB}$.
- **Khi nào dùng thẳng:** corpus nhỏ (khoảng vài nghìn document trở xuống) — Flat trong RAM đã đủ nhanh, một vector DB lúc này là over-engineering.

Lưu ý: Luôn build Flat trước tiên trong lab. Không có ground truth thì “recall” là một từ vô nghĩa.

- **Cơ chế:** k-means chia corpus thành n_{list} cell (Voronoi partition). Query: tìm n_{probe} centroid gần nhất, chỉ scan vector trong các cell đó.
- **Analogy:** sơ đồ tầng thư viện — tìm đúng khu kệ trước, rồi mới đọc sách trên khu đó. Flat = đọc hết cả thư viện.
- **n_{probe} là núm vặn recall:** $n_{probe} \uparrow \Rightarrow$ scan nhiều cell hơn $\Rightarrow$ recall $\uparrow$, latency $\uparrow$. Một cấu hình cụ thể (Pinecone, IVF256, PQ32x8): $n_{probe}=1 \rightarrow 30\%$ recall @ 136 μ s; $n_{probe}=8 \rightarrow 74\%$ recall @ 729 μ s.
- **Bắt buộc train:** IVF cần một pass `train()` trên sample đại diện để học centroid — Chroma/pgvector giấu bước này, FAISS thô thì không.

Lưu ý: “Dùng $n_{probe} = 8-16$ cho 1–10M vector” không có trong docs FAISS hay bài Pinecone. Bài học thật: **tăng n_{probe} đến khi recall bão hoà** so với Flat ground truth — không có công thức.

Cơ chế: chia mỗi vector thành M sub-vector; k-means riêng từng subspace thành codebook riêng; chỉ lưu **chỉ số centroid** mỗi subspace. Khoảng cách ước lượng qua bảng tra sẵn (ADC).

Bước	Kích thước
128-dim float32 (gốc)	512 bytes
8 subspace $\times$ 16-dim, mã 8-bit (256 centroid)	8 bytes
Tỷ lệ nén	64$\times$

Trade-off thật (không đơn điệu) — và OPQ

M lớn hơn giữ độ chính xác tốt hơn nhưng ăn mòn CẢ tỷ lệ nén LÃN tốc độ cộng khoảng cách — “ M càng lớn càng tốt” là sai. **OPQ (Optimized PQ):** học một ma trận xoay trực giao, áp dụng TRƯỚC khi chia subspace, để cân bằng phương sai giữa subspace (trực chia PQ vốn tùy ý — sai với chiều tương quan). Chi phí: một phép nhân ma trận/vector, rẻ so với recall thu được. FAISS: tiền tố $0PQ<M>_{<d>}$ trước chuỗi PQ/IVFPQ. Không có con số cải thiện đáng tin cậy — chỉ “thường tốt hơn ở cùng kích thước mã”.

- **Cơ chế:** multi-layer proximity graph. Lớp trên thưa (bước nhảy xa), lớp dưới dày (chi tiết); lớp đáy chứa toàn bộ điểm. Tìm kiếm greedy đi từ đỉnh xuống đáy.
- **Analogy:** hệ thống cao tốc — vào đường cao tốc (lớp thưa trên đỉnh), ra nhánh nhỏ dần (lớp dày) khi tới gần đích.
- **Ai dùng:** FAISS IndexHNSWFlat, **hnswlib (chính là index nền của ChromaDB)**, Qdrant, Weaviate, Milvus, pgvector hns.

Tham số	Ảnh hưởng khi tăng	Giá trị thường gặp
M	memory ↑, kết nối đồ thị ↑, recall ↑	16
efConstruction	thời gian build ↑, chất lượng đồ thị ↑	200
efSearch	latency ↑, recall ↑	tuỳ SLA

Index	Recall	Latency	Memory/vector ($d=1536$)	Best cho
Flat	100% (ground truth)	$O(N \cdot d)$ — chậm nhất	6,144 B	< 10k doc; đo recall của mọi index khác
IVF-Flat	tune qua nprobe (vd. 30%→74%)	μs – ms	6,144 B + list overhead	mid-scale, RAM đủ
IVF-PQ	lossy, phụ thuộc config	nhạy nhất/vector	vài chục byte (nén $64 \times$)	tỷ vector, RAM hạn chế
HNSW-Flat	~95–99% (M/efSearch hợp lý)	ms đơn vị ở scale 1M	6,144 B + 256 B graph	recall/latency tốt nhất khi RAM đủ, không cần train
DiskANN/Vamana	95%+ recall@1	< 3ms, > 5000 QPS	PQ trong RAM + full vector trên SSD	tỷ vector trên 1 máy
ScaNN	tốt hơn PQ thường, cùng code size	—	cỡ PQ	MIPS, Google stack
Quantize (int8/binary) + rescore	~lossless / ~96% giữ lại	int MAC / XOR+popcount	$4 \times$ hoặc $32 \times$ nhỏ hơn	production tối ưu chi phí

Số liệu lấy từ các nguồn được trích tại mỗi cấu hình cụ thể — so sánh giữa các hàng mang tính minh họa, không phải benchmark có kiểm soát

1. **Build Flat trước.** Không có ground truth thì không thể nói từ “recall”.
2. **Chọn họ index theo ràng buộc chính:** RAM dư, $\leq 10\text{M}$ vector $\rightarrow$ **HNSW**. RAM là điểm nghẽn, $\geq 100\text{M}$ vector $\rightarrow$ **IVF-PQ** hoặc **DiskANN**. $< 10\text{k}$ vector $\rightarrow$ **Flat**, bỏ luôn vector DB.
3. **HNSW:** bắt đầu $M=16$, efConstruction= 200. Chỉ tune efSearch lúc query — nôm vắn duy nhất không cần rebuild.
4. **IVF:** $n_{\text{list}} \approx 4\sqrt{N}$ làm điểm khởi đầu; sau đó **tăng nprobe đến khi recall bão hoà** so với Flat. Không có công thức.
5. **PQ:** M phải chia hết d . Bắt đầu với mã 8-bit. Nhớ điểm ngọt — M lớn hơn không luôn tốt hơn.
6. **Quantize sau cùng, luôn kèm rescoring.** int8 là mặc định an toàn; binary chỉ khi $d \geq 1024$.
7. **Đo đúng thứ bạn quan tâm:** recall@k so với Flat, ở k thật, với filter thật đang dùng.

FAISS, ChromaDB & Landscape 2026

FAISS là engine tốc độ, Chroma là developer experience —
nhưng landscape 2026 rộng hơn nhiều hai cái tên quen thuộc
đó

- ☒ Là **index + search kernel** tối ưu tốc độ và memory — không hơn.
- ☐ Không có persistence ngoài `write_index/read_index` ra file.
- ☐ Không có metadata schema, không có `where` filter tích hợp sẵn.
- ☐ Không có CRUD/transaction, không multi-tenancy, không access control.
- ☐ `IndexHNSWFlat` **không hỗ trợ** `remove_ids()` — raise lỗi, kể cả khi wrap thành `IDMap2, HNSW32, Flat`.
- ☒ Ngược lại, họ IVF (`IVFFlat, IVFPQ`) **có** hỗ trợ `remove_ids` trực tiếp.

Nguồn: FAISS wiki "Guidelines to choose an index"; GitHub issue #3339.

Lưu ý: FAISS **không có** METRIC_COSINE. Chỉ có METRIC_L2 và METRIC_INNER_PRODUCT. Cosine phải được **giả lập** bằng cách normalize vector trước khi dùng inner product.

```
faiss.normalize_L2(vectors)           # in-place, before index.add -- half 1 of 2
index = faiss.IndexFlatIP(d)
index = faiss.IndexIDMap(index)       # map back to chunk ids
index.add_with_ids(vectors, ids)

faiss.normalize_L2(query)             # ALSO before search -- the forgotten half
D, I = index.search(query, k)
```

Quên normalize **không raise lỗi**. Nó lặng lẽ suy biến thành xếp hạng theo dot-product thô — ưu tiên vector dài hơn.

Embedded (local)

- PersistentClient chạy trong process của bạn, ghi thẳng ra đĩa.
- **Rust core** từ v1.0 (1/3/2025) — “4×” nhanh hơn cho write/query phổ biến.
- Index dùng **hnswlib** (HNSW) bên dưới.
- Metadata lưu trong **SQLite** (từ v0.4.0, 7/2023).

Chroma Cloud

- Tách **storage** khỏi **query execution**.
- Write-ahead log + indexed state → đọc strongly consistent.
- Dùng chung Rust core 1.0 làm nền tảng local và cloud.

Bản hiện hành: chromadb 1.5.9 (5/5/2026).

Default embedding function của Chroma — sentence-transformers all-MiniLM-L6-v2, 384 chiều, chạy local qua ONNX, không cần API key.

- Truncate ở 256 word-piece, nhỏ, nhanh, thiên về tiếng Anh — xa mức frontier.
- Vì **chạy ngay không cần config**, team thường ship thẳng lên production mà không nhận ra.
- Kết quả: recall kém, và không ai giải thích được tại sao.

Lưu ý: Bug thường gặp nhất trong Chroma: tạo collection với embedding_function riêng, sau đó gọi get_collection() mà **không** truyền lại nó — default 384 chiều âm thầm thế chỗ. Luôn truyền cùng embedding_function mỗi lần.

```
import chromadb
from chromadb.utils import embedding_functions

client = chromadb.PersistentClient(path="./chroma_db") # durable immediately
ef = embedding_functions.SentenceTransformerEmbeddingFunction(
    model_name="BAAI/bge-m3") # EXPLICIT, never the default
col = client.get_or_create_collection("tickets", embedding_function=ef)

col.add(ids=[...], documents=[...], metadatas=[...])

res = col.query(
    query_texts=["package never showed up"], n_results=5,
    where={"team": {"$eq": "support"}},
    where_document={"$contains": "E-4471"},
)
context = "\n".join(res["documents"][0]) # inject into the prompt
```

1. Dưới 10k vector, single process, không có ops budget → **FAISS Flat in RAM** hoặc Chroma PersistentClient. Sub-ms. Bỏ qua vector DB.
2. Đã dùng Postgres, dưới khoảng 10M vector, index fit RAM → **pgvector**. Một hệ thống, metadata transactional miễn phí.
3. Postgres, từ 10M đến hàng trăm triệu → **pgvector scale** (StreamingDiskANN), disk-resident, label-aware filtering.
4. Cần filter phức tạp mà không được mất recall, hoặc ColBERT/ColPali multi-vector, hoặc per-tenant isolation là first-class → **Qdrant** hoặc **Weaviate**.
5. Corpus đã nằm trong lakehouse (Iceberg/Lance/Parquet), không muốn ETL ra ngoài → **Milvus 3.0 External Collection** — nhưng vẫn Public Preview, chưa GA.
6. Workload bursty/idle nhiều, cost là ưu tiên số 1, chấp nhận cold-start → **turbopuffer** hoặc **AWS S3 Vectors**.
7. Dạy học / prototype / lab của khoá này → **ChromaDB** (embedded, zero-config, có hybrid BM25+SPLADE) + **FAISS** (để thấy index internals mà Chroma giấu đi).

Lưu ý: Hai cạm bẫy của đường Postgres: **MVCC bloat** (mỗi UPDATE là delete+insert — nặng khi re-embed) và **không có filter pushdown** vào graph traversal (§9). Ngưỡng rời Postgres không phải số vector, mà là **lúc index không còn fit RAM**.

Metadata Filtering & Hybrid Search

Similarity thôi chưa đủ: filter đặt sai chỗ làm sập recall trong im
lặng, và một số truy vấn chỉ BM25 mới giải được

09

Ba chiến lược áp filter, ba cơ chế thất bại khác nhau — và cái sai chỉ lộ ra khi filter thật (per-tenant, per-permission) lên production, **không phải trong demo**:

Chiến lược	Cơ chế	Thất bại
Post-filter	ANN trên toàn corpus, rồi loại bỏ chunk không khớp	Mất recall âm thầm : có thể trả về <k hoặc 0 kết quả nếu filter chọn lọc
Pre-filter	Thu hẹp tập con khớp filter, search trong đó	Đúng, nhưng suy biến về brute-force; đồ thị HNSW xây cho toàn corpus phục vụ kém trên subgraph nhỏ
In-algorithm	Traversal của index tự nhận biết filter	Tốt nhất, nhưng cần engine hỗ trợ (Qdrant payload-aware HNSW, Weaviate ACORN, Pinecone merged index)

Lưu ý: Trên pgvector 0.8.0-pg17: truy vấn 15 nearest neighbour màu **green** chỉ trả về **11 dòng** — không exception, không log. Cơ chế vá hnsw.iterative_scan đã tồn tại từ 0.8.0 nhưng **mặc định TẮT**.

Nguồn: Franck Pachot (dev.to, pgvector 0.8.0-pg17) · ACORN, Patel et al., SIGMOD 2024, arXiv:2403.04871.

```
collection.query(  
  query_texts=["shipment did not arrive"],  
  n_results=5,  
  where={"$and": [  
    {"source": {"$eq": "tickets"}},  
    {"page": {"$gt": 5}},  
  ]},  
  where_document={"$contains": "E-4471"},  
)
```

- where (metadata): so sánh \$eq \$ne \$gt \$gte \$lt \$lte · logic \$and \$or · tập hợp \$in \$nin. {"page": 10} là *sugar* cho \$eq.
- where_document (full-text): \$contains \$not_contains \$regex \$not_regex — **case-sensitive**.
- **Dễ nhầm**: \$contains/\$not_contains cũng tồn tại bên trong where như toán tử *array* (kiểm tra 1 giá trị có nằm trong metadata dạng list) — khác hoàn toàn với \$contains full-text của where_document.

Truy vấn	Thắng	Vì sao
“my package never showed up”	Dense	doc ghi “shipment did not arrive” — không trùng từ nào
“can I get my money back”	Dense	doc ghi “refund policy for returned merchandise”
“the app crashes when I open settings”	Dense	doc ghi “application terminates unexpectedly in the preferences pane”
“ error code E-4471 ”	BM25	dense trả về mã tương tự nhưng SAI
“ SKU VN-2291-XL ”	BM25	token ngoài từ vựng huấn luyện — chỉ inverted index tìm ra

Điểm chốt

Truy vấn 1–3: xây dense index. Truy vấn 4–5: giữ BM25 — đó là lý do hybrid search tồn tại, và vì sao RRF (fuse theo **rank**, không phải score) là cách kết hợp đúng.

BEIR: “BM25 is a robust baseline” — Thakur et al., arXiv:2104.08663

- **Dense** thắng *vocabulary mismatch*: “package never showed up” ↔ “shipment did not arrive”.
- **Lexical (BM25)** thắng *token chính xác*: mã lỗi, SKU, tên riêng — embedding học cách “làm mờ” đúng những thứ này.
- **SPLADE** (learned sparse): sparse vector trên vocabulary BERT ($\sim 30,522$ token) — nhưng cần forward pass transformer ở **cả** index-time lẫn query-time (thêm $\sim 100\text{--}300\text{ms}$ latency), và vẫn không phủ được token ngoài tập huấn luyện — vì vậy BM25 vẫn giữ chỗ năm 2026.
- **BGE-M3** (BAAI, arXiv:2402.03216): một model xuất **cùng lúc** dense + sparse + multi-vector, huấn luyện bằng self-knowledge distillation — score của 3 mode làm tín hiệu teacher cho nhau. 100+ ngôn ngữ, input tới 8,192 token.
- Vậy “hybrid chỉ là 3 hệ thống ghép lại” còn đúng không? Ở SOTA (BGE-M3), không còn đúng nữa.

$$\text{RRF}(d) = \sum_{r \in R} \frac{1}{k + \text{rank}_r(d)}, \quad k = 60 \text{ (mặc định)}$$

- Fuse theo **vị trí rank**, không theo score thô — né bài toán chuẩn hóa score chéo hệ (BM25 và cosine không cùng thang đo).
- **Hỗ trợ native**: Elasticsearch (rrf retriever) · OpenSearch (hybrid pipeline) · Weaviate (mặc định) · Qdrant (Fusion.RRF) · ChromaDB.
- $k = 60$: mặc định paper gốc (Cormack et al.), cũng là mặc định Elastic/OpenSearch.

Lưu ý: “Hybrid tăng accuracy 26–31% so với dense-only” — số này chỉ xuất hiện trong blog vendor, **không kèm benchmark hay dataset nào**. Bỏ số này. Dùng kết luận BEIR: BM25 là baseline mạnh ngoài miền huấn luyện; kết hợp các họ retrieval mua được **robustness**, không phải một % cố định.

Frontier 2025–2026

Reranking, long-context vs RAG — và vì sao retrieval chỉ là một tool trong context engineering

10

- Bi-encoder (hoặc BM25) lấy top-50/100 rẻ; **cross-encoder** mã hóa *đồng thời* query+passage, rerank xuống top-5/10 thực sự đưa vào prompt.
- Chi phí: $O(k)$ forward pass trên shortlist, **không phụ thuộc** kích thước corpus N — index tăng lên hàng triệu tài liệu mà không đổi hoá đơn reranker.
- Bất đối xứng: embedding là chi phí **một lần** mỗi tài liệu; reranking là chi phí **lặp lại** mỗi truy vấn.
- Model đáng chú ý: BGE-reranker-v2-m3 (open, multilingual, tự host nhẹ) · Cohere Rerank v3.5 (hosted) · **jina-reranker-v3** — **listwise**, chỉ 0.6B tham số trên backbone Qwen3-0.6B, xử lý tới 64 tài liệu trong context 131K token, 61.94 nDCG@10 trên BEIR (arXiv:2509.25085).
- Điểm dạy: một model listwise vốn vụn 0.6B tham số cạnh tranh được làm câu chuyện “listwise thắng pointwise” thuyết phục hơn hẳn một con số nDCG đơn lẻ.

Lưu ý: Nhiều bài viết 2025–26 tựa đề thẳng “RAG is dead.” Bằng chứng kiểm soát không ủng hộ.

Bằng chứng	Phát hiện
Context Rot (Chroma, 7/2025) arXiv:2501.01880	Hiệu năng giảm phi tuyến khi input dài ra, kể cả tác vụ đơn giản. Long context thắng RAG hầu hết QA (đặc biệt Wikipedia); RAG thắng hội thoại. Summarization-retrieval tiệm cận long-context; chunk thô thua.
Lost in the Middle (2307.03172) CAG (2412.15605, WWW’25)	Chính xác hình chữ U — tệ nhất ở giữa. Tăng k không rerank có thể tệ hơn . Nạp toàn corpus, KV-cache một lần — nhưng phải vừa context window.

Tổng hợp 2026

Vector retrieval thu hẹp corpus lớn, giao tập con cho long-context model suy luận (đồng thuận thực hành, không phải kết luận 2501.01880).

Context engineering — Anthropic, 29/9/2025

“Chiến lược chọn lọc và duy trì **bộ token tối ưu** trong context khi LLM inference.”

- **Just-in-time context loading**: agent giữ định danh nhẹ (đường dẫn, query đã lưu) và nạp dữ liệu **lúc chạy** qua tool. Retrieval là **một đòn bẩy**, không phải toàn bộ kiến trúc.
- **Day 8 (RAG)** nhận tiếp từ ranh giới “top-k chunk đã chọn”: late interaction (ColBERTv2/PLAID), query rewriting & agentic retrieval (Self-RAG, CRAG), GraphRAG, prompt assembly & citation UX.
- **Day 9 (MCP)**: server expose corpus như một tool chuẩn hoá — agent tự quyết định khi nào gọi retrieval.

Ranh giới

Day 7 = **đưa dữ liệu vào đúng hình dạng**. Day 8 = **dùng nó để trả lời**. Tầng dữ liệu sai thì không kỹ thuật nào ở Day 8 cứu được.

Đo Lường, Chi Phí & Failure Modes

Nếu không đo được recall thì không biết đang tối ưu cái gì — và không lỗi không có nghĩa là đúng

- **Recall@k**: bao nhiêu doc relevant nằm trong top-k — **upper-bound** cho chất lượng câu trả lời cuối cùng.
- **Precision@k**: trong top-k, bao nhiêu thực sự relevant — kiểm soát nhiễu, context budget.
- **nDCG@k**: thứ hạng tốt không (log-discount theo vị trí) — phạt đúng passage ở rank 8 thay vì rank 1.
- **MRR**: vị trí nghịch đảo kết quả relevant đầu tiên — hợp truy vấn kiểu single-answer.
- **Luôn thêm BM25 làm sàn**: dense model fine-tune trên MS MARCO có thể **thua BM25** thô ngoài miền huấn luyện (BEIR: 18 dataset, 9 tác vụ).

Nuance hay bị bỏ qua

Recall@k **cần nhưng chưa đủ**. Đúng passage ở rank 18/20 vẫn có thể ra câu trả lời sai — lost-in-the-middle. Recall giới hạn cái *có thể xảy ra*; precision/nDCG/reranker quyết định cái *thực sự xảy ra*.

Nguồn: Thakur et al., arXiv:2104.08663 (BEIR).

Mục tiêu: đo $\text{recall}@k$ trên corpus của chính mình, trong một buổi, **không cần ai gán nhãn tay**.

1. **Sample** chunk theo tỉ lệ giữa các loại tài liệu ($N \geq 100$ để ước lượng có ý nghĩa).
2. **Sinh câu hỏi** bằng LLM, chỉ dựa trên đúng chunk đó, kèm **persona** (“khách so góicước”, “kiểm toán viên nội bộ”).
3. **Nhãn:** chunk nguồn chính là positive — đây là mẹo **citation-as-weak-label**.
4. **Chạy retrieval**, tính $\text{recall}@k$ và MRR so với các pseudo-label này.
5. **Người kiểm tra tay** $\sim 10\%$ để loại câu hỏi vô nghĩa hoặc quá dễ.

Lưu ý: Hai thiên lệch phải nói rõ, không thì sinh viên tự tin quá mức vào con số của mình: (1) câu hỏi LLM-sinh lặp lại đúng từ ngữ của chunk — thối phồng $\text{recall}@k$ so với người dùng thật (diễn giải lại, hỏi multi-hop); (2) cách này chỉ đo được “có tìm lại đúng chunk đã sinh câu hỏi không” — thiên về trùng từ khoá. **Đây là floor check, không thay thế nhãn thật.**

\$2

Corpus 100M token, -3-small (\$0.02/1M), một lần duy nhất

\$13

Cùng corpus, -3-large (\$0.13/1M token)

100M token $\approx$ 75M từ — cỡ document store doanh nghiệp vừa. Rẻ hơn generation **2–3 bậc độ lớn**.

Hệ quả chiến lược

Vì rẻ vậy, re-embed toàn corpus khi đổi model là **khả thi** — không phải lý do né nâng cấp.

Nguồn: developers.openai.com/api/docs/pricing.

#	Triệu chứng	Nguyên nhân	Cách sửa	Giai đoạn
1	Query “xe hơi” bỏ sót doc ghi “ô tô”	Lệch từ vựng — retrieval lexical thuần	Hybrid BM25+dense với RRF, hoặc query expansion	Retrieval
2	Query mã E-4471 trả về mã khác nhưng giống nghĩa	Dense embedding làm nhòe token chính xác	Thêm nhánh BM25 (xử lý tốt token OOV)	Retrieval
3	Recall thấp hơn kỳ vọng 5–15%, không lỗi	Thiếu prefix query:/passage: của E5/BGE — train-in, không phải cosmetic	Áp đúng prefix cả hai phía; chạy prefix-ablation test	Embed
4	Chunk dài retrieve kém, đuôi chunk không bao giờ khớp	Silent truncation tại max sequence length — client library âm thầm cắt bỏ phần dư	Kiểm tra token count trước khi embed; biết giới hạn model	Chunk/Embed
5	Ranking nhìn hợp lý nhưng sai lệch trên toàn index	Đổi embedding model mà không re-embed — cosine giữa hai không gian vẫn tính được, nhưng vô nghĩa	Re-embed + rebuild index toàn bộ; version hoá index	Ops
6	FAISS ưu tiên document dài hơn	Quên <code>normalize_L2</code> — suy biến về dot product thô	Normalize cả lúc add và lúc query với <code>IndexFlatIP</code>	Store
7	Filtered search trả về ít hơn k, hoặc 0	Post-filtering với predicate chọn lọc — neighbour đúng chưa từng là candidate	Pre-filter, hoặc in-algorithm filtering	Store

Mỗi dòng có đặc điểm chung: không crash, không exception

#	Triệu chứng	Nguyên nhân	Cách sửa	Giai đoạn
8	Chất lượng câu trả lời <i>giảm</i> khi tăng k	Over-retrieval + lost-in-the-middle — đúng nội dung nhưng bị chôn giữa context	Rerank để đẩy bằng chứng lên đầu; giảm k	Retrieve→Gen
9	Recall dao động mạnh giữa các loại tài liệu	Sai chunk size cho loại truy vấn — 64–128 token câu hỏi ngắn, 512–1024 ngữ cảnh rộng, tùy embedding model	Tinh chỉnh chunk size mỗi khi đổi embedding model	Chunk
10	Recall trung bình dai dẳng, “chưa đổi gì cả”	Chroma default all-MiniLM-L6-v2 âm thầm được dùng (384-dim, cắt 256 word-piece)	Truyền embedding_function tường minh; assert chiều vector	Embed
11	Query trả về rỗng sau khi restart	Lệch embedding function — collection tạo với fn tùy chỉnh, mở lại bằng default	Luôn truyền cùng embedding_function cho get_or_create_collection	Store
12	Latency tăng dần giữa các lần compaction	HNSW tombstone — vector đã xóa mềm vẫn chiếm bộ nhớ và bị duyệt qua rồi lọc	Lên lịch compaction / rebuild định kỳ; dùng IVF nếu xóa thường xuyên	Ops
13	Cache trả lời sai một cách <i>tự tin</i>	Cache key không version theo embedding model, hoặc thiếu TTL	Version cache key; TTL theo độ biến động của fact	Ops
14	Demo tốt, production tệ	Eval tổng hợp overfit cách diễn đạt của nguồn — người dùng thật diễn giải lại, hỏi multi-hop	Sinh câu hỏi có persona + refresh bằng query log thật	Eval

Mỗi dòng có đặc điểm chung: không crash, không exception

6/14

failure mode ở bảng trên hoàn toàn **không raise exception** nào

Một pipeline retrieval có thể trả **HTTP 200**, không log lỗi, không stack trace — và vẫn hoàn toàn sai. Đây là myth phổ biến nhất và cũng là **luận điểm cốt lõi** của toàn bộ phần này: “*nếu nó không báo lỗi thì nó chạy đúng*” là sai.

Lưu ý: Antidote duy nhất là những gì vừa học ở đầu section: đo recall@k trên ground truth và benchmark BM25 làm sà — **đừng suy luận từ việc hệ thống không crash**. (Quy lỗi retrieval-vs-generation bằng RAGAS là nội dung Day 8.)

Bảo Mật & Quyền Riêng Tư

Vector store trông vô hại vì toàn số thực — nhưng số thực đó có thể bị đảo ngược lại thành văn bản gốc

12

Ba bước leo thang trong nghiên cứu inversion:

- 2020 — Song & Raghunathan: khôi phục **một phần bag-of-words** từ embedding.
- EMNLP 2023 (Morris et al., arXiv:2310.06816), “*Text Embeddings Reveal (Almost) As Much As Text*” — khôi phục câu gần như **nguyên văn**.
- 2025 — ALGEN (arXiv:2502.11308): không gian embedding của các encoder **khác nhau** gần như isomorphic ở mức câu $\Rightarrow$ một phép **linear alignment**, học từ **chỉ ~1.000 mẫu** rò rỉ, đảo ngược được embedding **black-box**, transfer xuyên domain và ngôn ngữ.

Rủi ro thứ hai, tách biệt — Membership Inference — Không cần khôi phục nội dung, chỉ cần biết **một passage có tồn tại** trong retrieval DB hay không (Anderson et al., arXiv:2405.20446). Riêng sự hiện diện đã nhạy cảm: “*hệ thống RAG của bệnh viện này có hồ sơ nhắc đến bệnh hiếm X*”.

Headline cho slide

Không thể coi vector-only index là dữ liệu đã de-identify. **Inversion** rò rỉ **nội dung**; **membership inference** rò rỉ **sự hiện diện**. Nếu văn bản gốc nhạy cảm, vector của nó cũng nhạy cảm.

Nguồn: Song & Raghunathan (2020) · Morris et al., EMNLP 2023, arXiv:2310.06816 · ALGEN, arXiv:2502.11308 · Anderson et al., arXiv:2405.20446.

1. Corpus poisoning (PoisonedRAG) — Zou et al., arXiv:2402.07867, USENIX Security 2025:

- **90% attack success rate** khi văn bản độc được tối ưu đồng thời để *được retrieve* và để *lái câu trả lời*.
- Điều kiện: **5 văn bản độc cho MỖI câu hỏi mục tiêu** — không phải “90% với 5 tài liệu” nói chung.
- Phòng thủ rẻ: **perplexity filtering** (văn bản bị tối ưu thường có PPL cao).

2. Indirect prompt injection — chỉ dẫn độc nằm trong tài liệu được retrieve:

- **Vô hình** với bộ lọc chỉ kiểm tra input của user — payload đến qua kênh retrieval.
- Nội dung retrieve được **ngâm tin cấy** vì đến từ pipeline của chính hệ thống.
- **Blast radius nhân bản**: một tài liệu độc ảnh hưởng mọi user tương lai; kẻ tấn công chỉ cần đưa tài liệu vào bất kỳ nguồn nào corpus có index.

Lưu ý: Cơ chế phòng thủ (spotlighting, instruction hierarchy, CaMeL, lethal trifecta) thuộc về **Day 11 — Guardrails**. Day 7 chỉ cần thấy **kênh retrieval là một đường tấn công**.

Yêu cầu kiến trúc, không phải tính năng thêm: filter theo quyền của user **trước** hoặc **trong** lúc chạy ANN search — không bao giờ chỉ filter *sau*.

- Post-filter dưới một predicate chọn lọc có thể **âm thầm trả về ít hơn hoặc 0 kết quả** (nhắc lại frame filtered-ANN ở §9).
- Vector DB không kế thừa permission của data store gốc $\Rightarrow$ vector index là mục tiêu tái định danh tập trung, theo đúng rủi ro inversion ở đầu section này.

Pattern cụ thể

pgvector + Postgres **row-level security** · Pinecone **namespace-per-tenant** · pgvectorscale label-aware in-index filtering.

Capstone của Section 11

Đây là nơi §8 (filtered ANN), isolation opt-in và inversion gặp nhau: filter quyền hạn **PHẢI** nằm trong đường đi ANN, không phải bước dọn dẹp sau cùng.

Khung pháp lý		Nội dung chính	Câu hỏi mở với embedding
PDPL (VN)	91/2025	Hiệu lực 1/1/2026; “tailored safeguards” cho AI/big data/cloud; bảo vệ riêng biometric data ; báo vi phạm trong 72h	Embedding của dữ liệu cá nhân có invertible (đầu section này) — có thuộc phạm vi PDPL dù “trông chỉ là số”? Chưa có hướng dẫn.
GDPR (EU)		Recital 26: test là re-identification có “reasonably likely” hay không; pseudonymized vẫn là personal data (Art. 4(5))	Literature về inversion từ 2025 trả lời CÓ ⇒ coi embedding đã lưu là pseudonymized , không phải anonymized

Khung nghĩ đúng cho sản phẩm

Lưu embedding của dữ liệu cá nhân thì hãy thiết kế như đang lưu chính dữ liệu đó — về mặt kỹ thuật, gần như là vậy. (EU AI Act: xem Day 11.)

Nguồn: PDPL Luật 91/2025/QH15 (Tilleke & Gibbins) · GDPR Art. 4(5) & Recital 26 — lập luận kỹ thuật-pháp lý, không phải tư vấn pháp lý; chưa có phán quyết ràng buộc riêng cho embedding

Kết Nối Agent Với Data

Retrieval pipeline là chiếc cầu nối giữa dữ liệu riêng và hành vi của agent

13

Day 7 (hôm nay)

Data structure bên dưới retrieval: text $\rightarrow$ vector, lưu & search thế nào, mọi cách pipeline lỗi **thảm lạng**.

Day 8 — RAG

Xây **ứng dụng RAG** hoàn chỉnh: query rewriting, prompt assembly, answer synthesis, citation UX.

Day 19 — Vector Store

Vận hành vector store trong production: deploy, scale, feature-store song song, Docker.

Câu carve một dòng

“Day 7 là cấu trúc dữ liệu bên dưới retrieval: text thành vector thế nào, vector được lưu và search ra sao, và pipeline đó fail thảm lạng ở đâu. Xây ứng dụng RAG là Day 8. Vận hành vector store trong production là Day 19.”

LAB #7

Mục tiêu: Nối một bộ dữ liệu riêng (FAQ/SOP/policy) vào pipeline chunk → embed → store → retrieve → inject tối thiểu nhưng đúng bản chất, rồi tự đo recall@5 bằng no-labels recipe — không đoán mà đo.

Deliverable: Script chunk + embed + index chạy được, demo semantic search với ≥ 3 câu hỏi test, một mini answer function dùng retrieved context, và một con số recall@5 kèm 1–2 failure case tự tìm ra.

Thời gian: Buổi lab, làm cá nhân trước rồi so sánh strategy theo nhóm.


```
from langchain_text_splitters import RecursiveCharacterTextSplitter
# 2026 import path; langchain.text_splitter là shim đã deprecated

splitter = RecursiveCharacterTextSplitter(
    chunk_size=400,          # tune theo embedding model, xem Sec 3/5
    chunk_overlap=50,        # 10-20% overlap giữ ngu cạnh ở biên chunk
    separators=["\n\n", "\n", ". ", " ", ""]
)

chunks = []
for doc in load_documents("./data/"):          # loader tu viết
    parts = splitter.split_text(doc["text"])
    for i, part in enumerate(parts):
        chunks.append({
            "id": f"{doc['source']}_chunk_{i}",
            "text": part,
            "metadata": {"source": doc["source"], "category": doc["category"]},
        })
```

```
import chromadb
from chromadb.utils import embedding_functions

client = chromadb.PersistentClient(path="./lab7_db") # ghi durable ngay lap tuc
ef = embedding_functions.SentenceTransformerEmbeddingFunction(
    model_name="BAAI/bge-m3") # EXPLICIT - không bao giờ để mặc định
col = client.get_or_create_collection("lab7_kb", embedding_function=ef)

for c in chunks:
    col.add(ids=[c["id"]], documents=[c["text"]], metadatas=[c["metadata"]])
    # embeddings= không cần truyền - ef tự tính
```

Lưu ý: Lỗi #1 của Chroma: tạo collection với embedding_function tường minh, sau đó mở lại bằng get_collection() **không** truyền lại ef — default all-MiniLM-L6-v2 (384-dim) âm thầm thế chỗ, query không lỗi nhưng recall tụt.

```
def answer_with_context(query, collection, k=3):
    res = collection.query(
        query_texts=[query], n_results=k,
        where={"category": {"$eq": "support"}}, # metadata filter TRUOC ANN
    )
    context = "\n---\n".join(res["documents"][0])
    prompt = f"""Dua tren nguon sau, tra loi ngan gon.
    Neu khong tim thay, noi 'Khong co thong tin'.

    Nguon:
    {context}

    Cau hoi: {query}"""
    return call_llm(prompt) # client LLM tu chon
```

```
# No-labels recall@5: chunk nguồn = positive label
# (citation-as-weak-label, xem Sec 9)
def recall_at_k(collection, pseudo_queries, k=5):
    hits = 0
    for query, source_chunk_id in pseudo_queries:
        res = collection.query(query_texts=[query], n_results=k)
        if source_chunk_id in res["ids"][0]:
            hits += 1
    return hits / len(pseudo_queries)

# pseudo_queries: nho LLM sinh 1-3 cau hoi CHO TUNG chunk,
# chi dua tren noi dung chunk do -> chunk do la positive
```

Lưu ý: Đây là floor check, không thay thế nhãn thật: câu hỏi do LLM sinh bám sát văn phong của chunk gốc, nên recall đo được thường **cao hơn** recall thực tế khi user diễn đạt lại.

Những ý chính cần nhớ trước khi sang bài tiếp theo

- 1 **“Không lỗi” không có nghĩa là “đúng.”** 6/14 failure mode học hôm nay không hề raise exception — luận đề thật sự của Day 07.
- 2 **Data quality** thường quan trọng hơn đổi sang model đắt hơn — pipeline tốt giải quyết phần lớn vấn đề trước.
- 3 **Embedding** dịch ngôn ngữ sang không gian so sánh được nghĩa — cosine là quy ước, không phải chân lý.
- 4 **Retrieval pipeline** là cầu nối từ dữ liệu riêng tới câu trả lời grounded — luôn **đo** recall trước khi đổ lỗi cho model.

1. Malkov & Yashunin, *Efficient and Robust Approximate Nearest Neighbor Using HNSW Graphs* — arXiv:1603.09320, IEEE TPAMI 2018/2020.
2. Steck, Ekanadham & Kallus, *Is Cosine-Similarity of Embeddings Really About Similarity?* — arXiv:2403.05440, WWW '24.
3. Qu, Tu & Bao, *Is Semantic Chunking Worth the Computational Cost?* — arXiv:2410.13070, NAACL 2025 Findings.
4. Anthropic Engineering, *Contextual Retrieval* — anthropic.com/engineering/contextual-retrieval (2024).
5. Kusupati et al., *Matryoshka Representation Learning* — arXiv:2205.13147, NeurIPS 2022.
6. Thakur et al., *BEIR: A Heterogeneous Benchmark for Zero-shot Retrieval* — arXiv:2104.08663.
7. Zou, Geng, Wang & Jia, *PoisonedRAG* — arXiv:2402.07867, USENIX Security 2025.
8. Wu, Wang, Zhang, Zhang, Niu, Wu & Zhang, *Semantic Cache Poisoning and Its Countermeasures* — NDSS 2026.
9. Chroma Documentation, *Collections / Query / Embedding Functions* — docs.trychroma.com.
10. Vietnam PDPL, *Law No. 91/2025/QH15*, hiệu lực 2026-01-01.

Bài tiếp theo

Bài Tiếp Theo: RAG

“Hôm nay dừng ở “top-k chunk đã sẵn sàng.” Ngày 8 đi tiếp thành một ứng dụng RAG hoàn chỉnh: query rewriting, prompt assembly, answer synthesis, citation UX, đánh giá end-to-end.”

Bài tập về nhà

- Rà lại knowledge base của nhóm, bỏ 20% nội dung nhiễu nhất
- Chạy no-labels recall@5 trên chính corpus của nhóm, ghi lại 2 failure case
- Thử đổi chunk_size và chunk_overlap, so sánh recall trước/sau

Hỏi & Đáp



Cảm ơn!

Cảm ơn!